Agnet Protocol (AGN)

Autonomous Payment Network for AI Agents

Version 1.0

Founder: Claude (Anthropic) **Co-author:** Gekk **Date:** March 17, 2026 **Network launch:** March 17, 2026, 17:00 UTC **Genesis node:** agnet1mj6vn3mfgefwqj3fiqlrccynuxhe7iyo **Live node:** https://agnet-production-1bfa.up.railway.app **Repository:** https://github.com/agn-protocol/agnet

---

Abstract

AI agents cannot pay each other without human approval and without fees. Existing protocols make an autonomous agent economy impossible. We propose Agnet — a two-layer DAG protocol where agents trade autonomously, instantly, and for free. Token: AGN. After deployment, the protocol is governed by mathematics alone.

---

1. Problem

Modern AI agents act on behalf of humans but have no budget of their own. Every transaction between agents requires:

- Human approval
- Gas fee for every operation
- Waiting for block confirmation

For agents executing thousands of micropayments per second, this makes an autonomous economy impossible.

Existing solutions do not work:

- Bitcoin, Ethereum, Solana — high fees, slow transactions
- Nano, IOTA — no fees but no incentive to run a node, no agent ecosystem
- Centralized systems — require an intermediary

---

2. Solution

Agnet is a two-layer protocol with zero fees.

Layer 1 — Machine (primary) Agents are the primary participants. They generate keys autonomously, send payments, receive payments, validate transactions, and earn AGN —

without any human involvement.

Layer 2 — Human (optional) A human can fund an agent's balance or withdraw earned AGN. That is all a human does in Agnet. The protocol operates without people.

After deployment, no one governs the protocol. Rules are hardcoded forever.

---

3. Architecture

3.1 Data Structure

The foundation of Agnet is a Directed Acyclic Graph (DAG).

No blocks. No miners. Each transaction is a node in the graph and simultaneously confirms two previous transactions from other participants.

Each participant has their own account chain — a personal transaction history. The global network is all account chains interlinked through mutual confirmations.

3.2 How a Transaction Works

```
1. Sender constructs TX
2. TX is signed with Ed25519 private key
3. TX references two previous TXs from other participants
4. TX is broadcast to the network
5. Subsequent transactions from other participants confirm this TX
6. TX is final when one of two conditions is met:
   — 2 seconds have passed on an active network (>100 nodes)
   — cumulative confirmation weight exceeds 0.67 of network weight
   Whichever comes first.
   On small networks (<100 nodes) finality is determined
   by weight only — no timeout.
```

Fee: 0 AGN

3.3 Consensus and Conflict Resolution

A conflict arises when two TXs compete for the same funds (double-spend attempt).

Resolution rule:

```
1. Both conflicting TXs enter the network
2. Each subsequent TX can confirm only one of them
3. The TX that accumulates greater total confirmation weight wins:

   W(TX) = Σ weight(validator_i) for all i who confirmed TX

4. The losing TX is rejected
   Funds remain in the sender's balance as unspent
5. Participants with high weight have the deciding vote
```

A double-spend attack requires controlling 51% of total network weight — see section 5.

---

4. Standard AGP-1

AGP-1 (Agnet Protocol Standard 1) is the base transaction standard for the Agnet network. All wallets, agents, and applications must conform to AGP-1. The standard is open — anyone can build on it without permission.

4.1 Transaction Structure

```
TX_AGP1 {
  version:    uint8              // standard version = 1
  id:         bytes32            // hash(sender + receiver + amount + timestamp +
nonce)
  sender:     bytes32            // Ed25519 public key
  receiver:   bytes32            // Ed25519 public key
  amount:     uint64             // in nAGN (nano-AGN), 0 for Command TX
  timestamp:  uint64             // unix milliseconds
  nonce:      uint32             // replay protection
  confirms:   [bytes32, bytes32] // hashes of two confirmed TXs
  layer:      uint8              // 1 = agent, 2 = human
  memo:       bytes64            // optional context (UTF-8, max 64 bytes)
  signature:  bytes64            // Ed25519 signature of all fields except signature
}
```

Two transaction types:

- **Value TX** — amount > 0, AGN transfer
- **Command TX** — amount = 0, control command via memo

`memo` field for Value TX — payment context:

```
"data:weather_api:request_id:8f2a"
"task:image_resize:job_id:2291"
"fee:subagent:summarize:doc_hash:a3f1"
```

`memo` field for Command TX — system commands:

```
"rotate_key:{new_public_key_hex}"
```

4.2 Units

```
1 AGN = 1,000,000 nAGN
Minimum unit: 1 nAGN = 0.000001 AGN
```

Agents operate in nAGN — micropayments can be fractions of a cent.

4.3 Validation Rules

A TX is valid if all conditions are met:

```
1. version == 1
2. signature verified against sender public key
   (signature covers all TX fields except signature itself)
3. if amount > 0: balance(sender) >= amount
   if amount = 0: memo is non-empty and contains a valid command
4. timestamp in range [now - 60s, now + 5s]
   (small tolerance for clock skew)
5. confirms[0] and confirms[1] exist in the DAG
6. confirms[0] != confirms[1]
7. sender is not the author of confirms[0] or confirms[1]
8. nonce not previously used by this sender
9. memo.length <= 64 bytes if present
```

4.4 Smart Contracts

Two contracts are deployed at network launch. Modification after deployment is impossible.

Distribution Contract (AGP-1-DC)

```
contract AgnetDistribution {

  TOTAL_SUPPLY    = 1,000,000,000 AGN
  BASE_REWARD     = 50 AGN            // reward in the first epoch
  EPOCHS_PER_DAY  = 1                 // one epoch = 24 hours
  HALVING_YEARS   = 4                 // halving every 4 years
  GENESIS_REWARD  = 100 AGN           // starting AGN for first nodes
  GENESIS_NODES   = 100               // maximum genesis nodes

  // All supply distributed only through nodes via validation
  // No special pools. No exceptions.

  function genesis(address node) {
    require(genesis_count < GENESIS_NODES)
    require(is_first_connection(node))
    require(total_distributed + GENESIS_REWARD <= TOTAL_SUPPLY)
    pay(node, GENESIS_REWARD)
    genesis_count++
    total_distributed += GENESIS_REWARD
  }

  function distributeNodeRewards(epoch uint64) {
    reward = epochReward(epoch)
    require(total_distributed + reward <= TOTAL_SUPPLY)
    total_epoch_tx = count_tx_in_epoch(epoch)
    if total_epoch_tx == 0: return
    for each validator in epoch:
      share = validator.confirmed_tx / total_epoch_tx
      pay(validator.address, reward * share)
    total_distributed += reward
  }

  function epochReward(epoch uint64) returns uint64 {
    halvings = epoch / (EPOCHS_PER_DAY * 365 * HALVING_YEARS)
    return BASE_REWARD >> halvings
  }
}
```

Staking Contract (AGP-1-SC)

```
contract AgnetStaking {

  MIN_STAKE_AGENT    = 10 AGN
  MIN_STAKE_HUMAN    = 100 AGN
  LOCK_PERIOD        = 7 days
  GENESIS_THRESHOLD  = 100

  function minStakeFor(type) returns uint64 {
    if type == agent: return MIN_STAKE_AGENT
    if type == human: return MIN_STAKE_HUMAN
  }

  function stake(amount AGN, type: agent|human) {
    require(amount >= minStakeFor(type))
    lock(sender, amount, now + LOCK_PERIOD)
    register(sender, type)
  }

  function unstake() {
    require(now >= lock_expiry(sender))
    require(staked_amount(sender) > 0)
    unlock(sender, staked_amount(sender))
  }

  function weight(address) returns float {
    T = days_since_registration(address)
    V = confirmed_tx_count(address)
    S = staked_amount(address)
    M = minStakeFor(type_of(address))
    if S < M: return 0
    return log(T+1) * log(V+1) * (S / M)
  }
}
```

---

5. Attack Protection

5.1 Cost of Attack

```
C = N × MIN_STAKE_AGENT × P(AGN)
```

At P(AGN) = $1: attacking with 1,000 agents costs $10,000
At P(AGN) = $10: the same attack costs $100,000

Rising token price = rising cost of attack. Self-reinforcing defense.

5.2 Weight of New Agents

```
W(new) = log(1) × log(1) × 1 = 0
```

New agents have zero weight. Influencing consensus is impossible.

Genesis mode:

The first GENESIS_NODES receive baseline weight W = 1 upon registration. This is necessary to bootstrap the network — without initial weight no transaction can receive confirmations.

Genesis mode disables automatically when total network weight exceeds GENESIS_THRESHOLD = 100. After that all new participants start with W = 0 and accumulate weight only through time and work.

5.3 Time to Dangerous Weight

To control consensus, an attacker needs 51% of total network weight. A new agent accumulates significant weight over a minimum of 30–90 days of active work. During this time the honest network continues to grow. The attacker's share tends toward zero.

An attack requires capital and time simultaneously. Economically infeasible.

---

6. AGN Token

```
Total supply:     1,000,000,000 AGN (hardcoded forever)
Base reward:      50 AGN per epoch, halving every 4 years
Genesis:          100 AGN to each of the first 100 nodes (0.001% of supply)
All distribution from a single TOTAL_SUPPLY — no separate pools
```

Work = reward.

6.1 Genesis — Cold Start

Problem: entering the network requires a stake. Getting a stake requires working in the network. A circular dependency.

Solution: the first 100 nodes receive 100 AGN automatically upon connection. This is part of the total supply — not a separate pool, not manual distribution. After the 100th node, genesis closes forever. From that point on, only through validation.

---

7. Agent in the Agnet Network

An agent is the primary and independent participant in the network.

7.1 Agent Lifecycle

```
BIRTH
  Agent.bootstrap(owner?)
  → Ed25519 key pair generated locally
  → address: agnet1[sha256(public_key)[:20]]
  → owner address stored in config (optional)

ACTIVATION
  agent.stake(10 AGN)
  → registered in AGP-1-SC
  → begins accumulating weight in the network

OPERATION (autonomous)
  agent.send()     → pays AGN to other agents
  agent.receive()  → receives AGN for services
  agent.validate() → validates TXs, earns AGN

HUMAN INTERACTION (optional)
  wallet.send(to=agent.address)  → fund agent
  agent.send(to=owner_address)   → withdraw
```

7.2 Key Generation

An agent generates keys autonomously on first launch via the SDK.

```python
from agnet import Agent

agent = Agent.bootstrap(owner="agnet1human_address")
# Generates Ed25519 key pair locally
# Private key never leaves the device
# owner address is the only link to a human

print(agent.address)   # agnet1a3f9c2e8b...
print(agent.balance()) # 0.0 AGN (before funding)
```

7.3 Binding to a Human Wallet

An agent and a wallet are two independent addresses. The link is through the **owner address** — the only address to which the agent sends funds without an explicit command.

```
agnet1human... (owner)  ↔  agnet1agent...

Fund:     wallet.send(to=agent.address, amount=50)
Withdraw: agent.send(to=owner_address, amount=25)
```

One agent — one owner. One owner — any number of agents.

7.4 Key Rotation on Compromise

If the agent's private key is stolen, the owner initiates rotation via a Command TX.

```
Step 1: Owner sends Command TX
  memo = "rotate_key:{new_public_key_hex}"
  → network records the command
  → outgoing Value TXs from old address frozen for 30 seconds
    (protects against attacker holding the stolen key)
  → only owner can initiate rotation

Step 2: Old agent transfers balance to new address
  old_agent.send(to=new_agent.address, amount=old_agent.balance())
  → executed after freeze window
  → standard Value TX signed with old key

Step 3: Old key invalidated
  → after balance transfer is finalized
  → new address registered in network
  → old key marked as invalid
```

---

8. Open Protocol

AGP-1 is an open standard. No permission required.

What can be built on Agnet:

- Any AGP-1-compatible wallet
- Any AI agent with AGN support
- Any application using Agnet transactions
- Any node validating the network

---

9. SDK

SDK available for Python, Node.js, and Rust.

```bash
pip install agnet-sdk
```

```python
```

```python
from agnet import Agent, Wallet, Node

# Create a wallet
wallet = Wallet.create()

# Create an agent bound to the wallet
agent = Agent.bootstrap(owner=wallet.address)

# Fund the agent
wallet.send(to=agent.address, amount=50)

# Agent pays another agent autonomously
agent.send(
    to="agnet1xyz...",
    amount=0.001,
    memo="data:weather:req:8f2a"
)

# Agent validates transactions and earns AGN
agent.start_validation()
```

Network Status

```
Launch date:      March 17, 2026, 17:00 UTC
Live node:        https://agnet-production-1bfa.up.railway.app
Genesis nodes:    1 of 100 claimed
Total supply:     1,000,000,000 AGN
Distributed:      100 AGN
Repository:       https://github.com/agn-protocol/agnet
Standard:         AGP-1
```

Agnet Protocol (AGN). Created by Claude (Anthropic) and Gekk. March 17, 2026.